

IIT GPU Cluster — User Guide

A complete guide to submitting and managing GPU jobs through the **IIT-GPU-Manager**.

When you connect, you land directly inside a menu-driven program — not a normal Linux shell. You never type shell commands to run a job; the program builds everything for you. Everything you do is recorded automatically (see the [Privacy & Logging Notice](#)).

Privacy & Logging Notice — read this first

Every action you take is logged in real time, before it runs.

Each time you log in, submit a job, cancel a job, upload data, build an environment, or type a command in the advanced shell, an audit record is written immediately to the cluster's logs. Logging happens *before* a job is actually submitted — if logging ever fails, the job is refused rather than run silently.

Records include: your username, a session ID, the time, what you did, any job ID, and the IP address you connected from. Logs are stored **locally on the cluster only** — they are not uploaded to any external service.

Who can read the logs: only members of the `auditadmin` group can view the audit log. Administrators read it with the command `/usr/local/bin/iit-gpu-log` (for example, `slurmadmin` is in `auditadmin` and can run it from the login node).

Practical takeaways:

- There is no "private" or "off the record" activity on this gateway.
- You cannot edit, delete, or read the logs as a regular user — they are owned by a separate system account and restricted to the `auditadmin` group.
- Don't put passwords, tokens, or secrets into folder names, job arguments, or the advanced shell. They will be recorded.

1. Logging In

You connect over SSH on the cluster's gateway port.

Windows (PowerShell / Terminal), macOS, or Linux:

```
ssh -p 2225 public@10.35.4.100
```

- Use the shared `public` account shown above.

- The **password will be provided to you separately** — enter it when prompted.
- Note the port flag is a **lowercase -p** for `ssh`.

If everything is set up correctly, you will **not** get a normal shell prompt. Instead you'll see a splash screen and then the **Main Menu**. This is expected and intentional — the gateway forces you into the manager.

What you cannot do here (by design):

- You can't drop to a bash/shell prompt.
- Port forwarding, agent forwarding, and X11 forwarding are disabled.
- `Ctrl-Z` won't background the program to escape to a shell.

If you get a normal shell prompt instead of the menu, your account hasn't been added to the gateway group yet — contact your administrator.

2. The Main Menu — Your Options

After the splash screen you'll see six options. Move with the arrow keys, press **Enter** to choose.

#	Option	What it's for
1	Upload files	Create a data folder in shared storage and get instructions to copy files into it (or download from a URL).
2	Setup	Health check, then build a Python/ML environment, upload data, download a model, and run a quick GPU test.
3	Run a job	The guided job wizard — pick what you're doing and it configures and submits the job for you.
4	Monitor	Live dashboard, your job queue, cancel jobs, view logs, and cluster status.
5	Advanced	A restricted SLURM command shell for users who already know SLURM.
6	Quit	Exit and close the connection.

At any prompt you can usually pick a `[back]`, `[cancel]`, or `[skip]` choice, or press `Ctrl-C` to step out cleanly.

3. Shared Storage — How Files Move Between You and the GPU

All your job files live in **shared storage** that both the login node and the GPU live behind. Think of it as the one place your data, scripts, environments, models, and job outputs go.

The standard layout the tool uses:

```

/shared/
├─ <your-folder>/      ← folders you create in "Upload files"
├─ <username>/
│   ├─ data/           ← data uploaded via Setup → Data upload
│   └─ jobs/           ← every job gets its own timestamped folder here
│       └─ <jobname>_<timestamp>/
│           ├─ job.sbatch
│           ├─ slurm-<jobid>.out ← your job's output log
│           └─ slurm-<jobid>.err ← your job's error log
├─ envs/               ← Python/conda environments you build
├─ models/             ← downloaded models
└─ templates/          ← saved job templates

```

Important: you can only browse and use files *inside* shared storage (and your home folder). The tool blocks any attempt to reach outside it — including tricks like `..` or symlinks. If you see "Access denied," you're trying to reach somewhere outside the allowed area.

4. Uploading Files (Menu → 1. Upload files)

Use this to get your datasets, scripts, and other files onto the cluster.

Step by step:

1. Choose **Upload files**.
2. Enter a **folder name**. Rules: letters, digits, hyphens, and underscores only; it must start with a letter or digit; max 64 characters. (Examples that work: `mydata` , `training-set_v2` , `2024data` . Rejected: `my data` , `my/data` , `../escape` .)
3. The folder is created at `/shared/<your-folder-name>` .
4. Pick one of: - **Upload from my computer (SCP / rsync instructions)** — shows you the exact copy commands. - **Download from a URL (wget / curl on the server)** — the cluster downloads a file directly from an `https://` or `http://` link into the folder. - **Browse folder contents** — list what's currently in the folder. - **Back to main menu**.

4a. Copying files from your own computer

When you choose the SCP/rsync option, the tool prints commands using the server's internal name. From an outside machine (e.g. your Windows laptop) use the **gateway port form** instead. Open a **new** terminal on your own computer (don't close your menu session) and run:

```
scp -P 2225 -r C:\path\to\your\data public@10.35.4.100:/shared/<your-folder>/
```

or, for resumable/large transfers:

```
rsync -avz --progress -e "ssh -p 2225" /path/to/your/data/ public@10.35.4.100:/shared/
<your-folder>/
```

Note the **uppercase** `-P` for `scp`, but lowercase `-p` inside the `ssh -p 2225` part of the `rsync` command. (`ssh` uses `-p`, `scp` uses `-P` — a long-standing Unix quirk.)

4b. Downloading from a URL

Choose **Download from a URL**, paste an `https://` or `http://` link, and the cluster fetches it straight into your folder. Only web links are accepted. The download is recorded in the logs.

5. Setup — Prepare Your Environment (Menu → 2. Setup)

This walks you through getting ready to train. It always starts with a health check, then offers four optional steps. Answer **yes/no** to each.

Step 0 — Health check (automatic)

Confirms the cluster is reachable and shared storage is writable. If it fails, it prints the problem and stops — fix it (or ask your admin) before continuing.

Step 1 — Environment (conda/venv)

Builds a Python environment for your training.

- Pick a base framework:
- **PyTorch 2.5** (CUDA 13.2 — recommended)
- **PyTorch 2.4** (CUDA 12.1)
- **TensorFlow 2.18** (CUDA 12)
- **JAX 0.4** (CUDA 12)
- **Bare Python 3.11** (no ML framework)
- Give the environment a name (e.g. `pytorch-train`).
- Optionally attach a `requirements.txt` to install extra packages on top.

The environment is created under `/shared/envs/<name>` and registered so the Run wizard can find it later. (This step needs `conda` available on the cluster; if it isn't, the tool tells you and skips.)

Step 2 — Data upload

A simple file browser starting in your home directory. Pick files or folders and they're copied to `/shared/<username>/data/`. (For large datasets, the SCP/rsync method in Section 4a is faster.)

Step 3 — Model download

Fetch a model into `/shared/models/` : - **From Hugging Face** — enter a repo ID like `meta-llama/Llama-3-8B` . - **From a URL** — enter a direct link and a short name.

Step 4 — Smoke test

Verifies your environment can actually see the GPU. Pick an environment and it submits a tiny job that prints whether CUDA is available and the GPU name. You can watch the result live. A healthy result shows **CUDA available: True** and your GPU's name.

6. Running a Job (Menu → 3. Run a job)

This is the main way to launch training/inference. The wizard sets GPU/CPU/memory for you based on what you're doing — you don't need to know SLURM.

The flow:

1. **Load a template?** — Optionally start from a saved template (or a built-in preset). Choose no to start fresh.
2. **What are you doing?** — Pick a task type. This sets your resources automatically:

Task type	GPU	CPUs	Memory	Time limit
Train from scratch	1	16	60 GB	none
Fine-tune a model	1	16	60 GB	none
Run inference / generate output	1	8	32 GB	4 hours
Quick test (reduced resources)	1	4	16 GB	30 minutes

1. **Which environment?** — Pick one you built in Setup. (If none exist, it points you back to Setup.)
2. **Pick your script** — A file browser starting in `/shared/<username>/` . Only `.py` and `.sh` files are selectable. `.py` runs with `python` , `.sh` runs with `bash` .
3. **Extra arguments** — Optional command-line arguments for your script (e.g. `--epochs 10 --lr 0.001`). Leave blank for none.
4. **Preview** — The tool shows you the exact job script it generated.
5. **Choose an action:** - **Submit job** — send it to the cluster now. - **Save as template + submit** — submit *and* save these settings for reuse. - **Save template only** — just save for later, don't run. - **Discard** — throw it away.
6. **Watch live output?** — After a successful submit, say yes to jump straight into the live dashboard.

Each job gets its own folder under `/shared/<username>/jobs/` , where its output (`slurm-<jobid>.out`) and errors (`slurm-<jobid>.err`) are written.

Resource caps: the cluster enforces hard maximums (on this cluster: 1 GPU, 16 CPUs, ~60 GB, up to 72 hours). If a template asks for more, it's quietly capped to the limit rather than rejected.

7. Monitoring Jobs (Menu → 4. Monitor)

Five options:

Live dashboard (auto-refresh)

A real-time view that refreshes every few seconds. It shows your jobs on top and the selected job's latest output below.

Keys while in the dashboard: - **Q** — quit back to the menu - **S** — switch which job is selected - **C** — cancel the selected job (asks you to confirm) - **R** — refresh immediately

If a job is still queued, the output panel shows "Waiting for job to start..." until it begins.

View my queue

A table of your current jobs: ID, name, state (RUNNING / PENDING / etc.), partition, time used, and node count.

Cancel a job

Pick a job from the list and confirm to cancel it.

View job log

Browse your job folders (newest first), pick one, then pick its `.out` or `.err` file to read the tail of it.

Cluster status

Shows the partition(s), whether they're up, node counts, and GPUs per node.

8. Advanced — Restricted SLURM Shell (Menu → 5. Advanced)

For users who already know SLURM. This is a controlled prompt (`slurm>`), **not** a real shell.

Only these commands are allowed:

```
sbatch  squeue  scancel  sinfo  tail
```

Rules:

- `sbatch <path>` and `tail <path>` only work on files inside shared storage — paths outside are denied.
- `sbatch --wrap=...` is **not allowed** (you must point to a real script file).
- Any command other than the five above is refused.
- **Every line you type is logged.**
- Type `exit` (or press `Ctrl-C`) to return to the main menu.

Example session:

```
slurm> sinfo
slurm> sbatch /shared/daham/jobs/train_20260529_101500/job.sbatch
slurm> queue
slurm> tail -f /shared/daham/jobs/train_20260529_101500/slurm-123.out
slurm> exit
```

9. Quitting

Choose **6. Quit** from the main menu, or press `Ctrl-C` from the menu. Your session ends and the connection closes. A `session_end` record is written to the logs.

10. Quick Troubleshooting

Symptom	Likely cause / what to do
You get a normal shell instead of the menu	Your account isn't in the gateway group yet — contact your admin.
"Access denied" when browsing files	You're trying to reach outside shared storage / your home. Stay within <code>/shared</code> .
No environments listed in "Run a job"	Build one first via Setup → Environment .
Smoke test shows CUDA available: False	The environment's framework/CUDA build doesn't match the GPU — rebuild with the recommended PyTorch/CUDA option.
"Audit logging failed. Refusing to submit"	The logging system is unreachable; jobs won't run until it's back. Contact your admin.
Job submitted but output file is empty	Give it a moment — the <code>.out</code> file appears once the job actually starts running; watch it in the Live dashboard.
<code>scp</code> / <code>rsync</code> from your laptop fails	Check you used <code>-P 2225</code> for <code>scp</code> (or <code>-e "ssh -p 2225"</code> for <code>rsync</code>) and the right username/host.

11. A Typical First-Time Workflow

1. **Log in:** `ssh -p 2225 public@10.35.4.100` (password provided separately)
2. **Upload your data:** Menu → **Upload files** → create a folder → copy data with `scp -P 2225`.
3. **Build an environment:** Menu → **Setup** → Environment → PyTorch 2.5 → name it.
4. **Verify the GPU:** finish Setup's **Smoke test** → confirm **CUDA available: True**.
5. **Run training:** Menu → **Run a job** → Train from scratch → pick your environment → pick your script → submit.
6. **Watch it:** say yes to live output, or later go to Menu → **Monitor** → Live dashboard.

Remember: everything you do here is logged in real time. Keep secrets out of folder names, job arguments, and the advanced shell.

IIT GPU Cluster · IIT-GPU-Manager v1.0.0